FICHE APPRENANT · PROJET SUIVI D'INCIDENTS BACKBONE

Semaine 8

Tests, finitions et soutenance

Durée : 5 jours Tests automatisés Qualité du code Documentation Présentation

Objectifs de la semaine

- écrire des **tests automatisés** qui protègent l'application ;
- formater et relire son code ;
- documenter le projet pour qu'un autre développeur puisse l'installer ;
- présenter son travail et défendre ses choix techniques.

1. Pourquoi tester ?

Sans tests, chaque modification peut casser quelque chose ailleurs sans qu'on le voie. Un test automatique vérifie en quelques millisecondes ce qu'il faudrait 10 minutes à refaire à la main.

```
php artisan make:test IncidentTest
php artisan test
```

2. Anatomie d'un test

```
class IncidentTest extends TestCase
{
    use RefreshDatabase;   // base vierge avant chaque test

    public function test_un_incident_resolu_exige_cause_et_solution(): void
    {
        // 1. Préparer
        $user = User::factory()->create();
        $incident = Incident::factory()->create();

        // 2. Agir
        $reponse = $this->actingAs($user)->put(route('incidents.update', $incident), [
            'titre' => $incident->titre,
            // ...
            'statut' => 'resolu',
        ]);

        // 3. Vérifier
        $reponse->assertSessionHasErrors(['cause_racine', 'solution']);
    }
}
```

Le schéma **Préparer / Agir / Vérifier** structure tous les tests.

Assertion	Vérifie que…
`assertOk()`	la page répond 200
`assertRedirect(route('login'))`	on est redirigé
`assertForbidden()`	l'accès est refusé (403)
`assertSessionHasErrors(['champ'])`	la validation a échoué sur ce champ
`assertSee('INC-2026')` / `assertDontSee(...)`	le texte est présent / absent
`assertDatabaseHas('incidents', [...])`	la ligne existe en base

3. Les états de factory

```
public function admin(): static
{
    return $this->state(fn () => ['role' => Role::Admin]);
}
```

`User::factory()->admin()->create()` ou `Incident::factory()->resolu()->create()` rendent les tests lisibles.

4. Les tests et la base de données

`phpunit.xml` configure les tests sur **SQLite en mémoire** :

```
<env name="DB_CONNECTION" value="sqlite"/>
<env name="DB_DATABASE" value=":memory:"/>
```

Les tests sont rapides et ne touchent jamais ta base MySQL de développement. Conséquence : évite le SQL spécifique à MySQL (`YEARWEEK` , `TIMESTAMPDIFF`) dans le code de l'application, ou isole-le.

5. Qualité du code

```
vendor/bin/pint       # formate tout le code selon les conventions Laravel
```

Relis ton code avec ces questions :

- une méthode fait-elle **une seule chose** ?
- les noms disent-ils **ce que** fait le code (`dureeLisible()` plutôt que `calc2()`) ?
- y a-t-il du code copié-collé à factoriser ?
- reste-t-il des `dd()` , `dump()` ou du code commenté ?

6. Le README

Un bon README répond à : *Qu'est-ce que c'est ? Que fait-il ? Comment l'installer ? Comment se connecter ? Comment lancer les tests ?* Teste-le en suivant tes propres instructions dans un dossier neuf.

Travail demandé

1. Écrire au moins **10 tests** :
   - redirection des invités vers la connexion ;
   - connexion réussie ;
   - toutes les pages principales répondent 200 pour un admin ;
   - la création génère une référence et une entrée de suivi ;
   - la résolution exige cause et solution ;
   - la résolution renseigne la date et trace le changement de statut ;
   - ajout d'un suivi ;
   - un technicien n'accède pas aux utilisateurs et ne supprime pas d'incident ;
   - les filtres de la liste ;
   - le rapport hebdomadaire ne prend que la semaine choisie.
2. Passer Pint sur tout le projet.
3. Rédiger le `README.md` (présentation, fonctionnalités, installation, comptes de démo, tests).
4. Relire l'application comme un technicien NOC et corriger 5 irritants (libellés, alignements, messages…).
5. Préparer une démonstration de 15 minutes :

- **scénario** : « 08h12, alarme LOS sur FO-BKO-SGU » → déclaration P1 → assignation → interventions dans le journal → résolution avec cause racine → apparition dans la fiche hebdomadaire ;
- **3 choix techniques** expliqués (enums, Form Request, scope…) ;
- **ce qui reste à améliorer**.

Livrable

- `php artisan test` entièrement vert ;
- README complet et testé ;
- démonstration réalisée.

Auto-évaluation

- [ ] Au moins 10 tests, tous verts
- [ ] J'ai volontairement cassé une règle métier et vu un test échouer
- [ ] Un collègue a installé le projet avec mon seul README
- [ ] Le scénario de démonstration se déroule sans bug
- [ ] Je sais justifier mes choix et citer 3 limites de mon application

Ressources

- Laravel — Tests HTTP : laravel.com/docs/http-tests
- Laravel — Tests et base de données : laravel.com/docs/database-testing
- Laravel Pint : laravel.com/docs/pint

Pistes d'évolution

- Pièces jointes sur un incident (photos terrain, logs) avec `Storage`.
- Calcul de la **disponibilité** mensuelle d'une liaison (99,9 % ?).
- Historique de toutes les modifications d'un incident.
- Notification des P1 par SMS ou e-mail.
- API JSON pour recevoir les alarmes de l'outil de supervision (Zabbix, LibreNMS…).